Tree:-

The tree is a non-linear data structure; in which we insert the data in a hierarchical structure.

In our project requirement, if any hierarchical data structure is required, it is highly recommended to use a tree data structure.

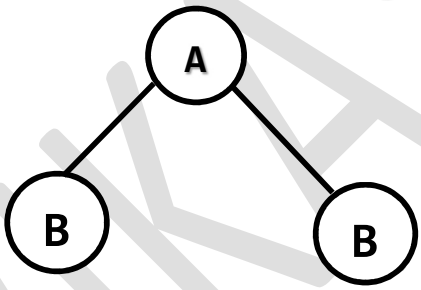
- **Tree is a non-linear data structure.**
- **Tree is a collection of elements or nodes.**
- **In the tree hierarchy top of the node is marked as the root node and the remaining node is divided into multiple subtree below the root.**

Uses of tree data structure:-

1. **Banking Application.**
2. **File System.**
3. **Family hierarchy.**

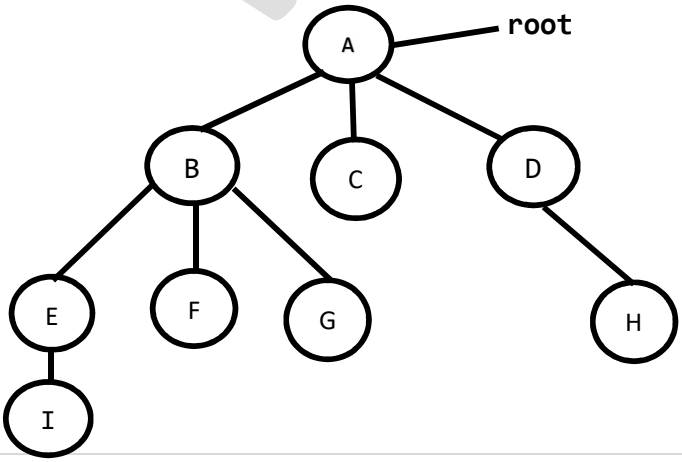
Basic Terminology in Tree Data Structure:-

Node:- Every individual element of a tree is known as node. Node storing a value and it connect or linked with other node.



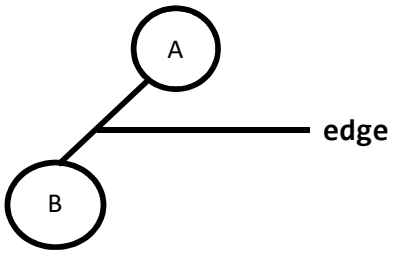
A B and C are node.

Root:-It is top of the node of tree hierarchy, they have no any parent.



VIKAS DS

Edge:- It is used to connect of two tree.



Path:- It is sequence of nodes from source node till destination node.

Example:- Path between A to G is A-B-C or A to B and B to G.

Leaf node:-The nodes which have no any child called leaf node.

Example:-C E F G H is leaf node.

Height or Depth of Tree:-The number of edge on longest path between root and leaf.

Example:- 3 is height or depth of above tree.

Level of tree:-In tree all nodes present in different level.

Root node level is 0(zero).

Direct child of root node level is 1(one).

Grand child of root node level is 2(two).

A----------level 0.

B C D----------level 1.

E F G H----------level 2.

Parent node and Child node:-In the tree hierarchy two nodes are connected to each other, the top hierarchy kwon as the parent node, and the bottom hierarchy known as the child node.

Example:- A is a parent node of BCD and B is a parent of EGF.

Sibling node:-If more than one node has the same parent node then it is sibling.

BCD is sibling node.

EFG is sibling node.

VIKAS DS

Ancestor node: - A node reachable through repeated moving from child to parent.

Example:- E B A where BA is an ancestor of E and A is an ancestor of B.

Degree of node:-The degree of a node is the number of subtrees coming out of a node.

Example:-Degree of the B node is 3, degree of the C node is 0.

Degree of tree node:-The maximum degree of a node is called as the degree of the tree.

Example:-Degree of the above tree is 3.

Predecessors node:-When we display or traverse a tree and a node came before some other node then that node is known as the predecessor's node.

Example:-B is the predecessor of E.

Successor node:- When we display or traverse a tree and a node came next to some other node then that node is known as the successors' node.

Example:- E is the successor of B.

Types of Tree:-Tree are divided into different parts based on properties and characteristics.

1. **Binary Tree.**
2. **Binary Search Tree.**
3. **AVL Tree.**
4. **B-Tree.**
5. **B+ Tree.**
6. **Red-Black Tree.**
7. **Segment.**

Binary Tree:-

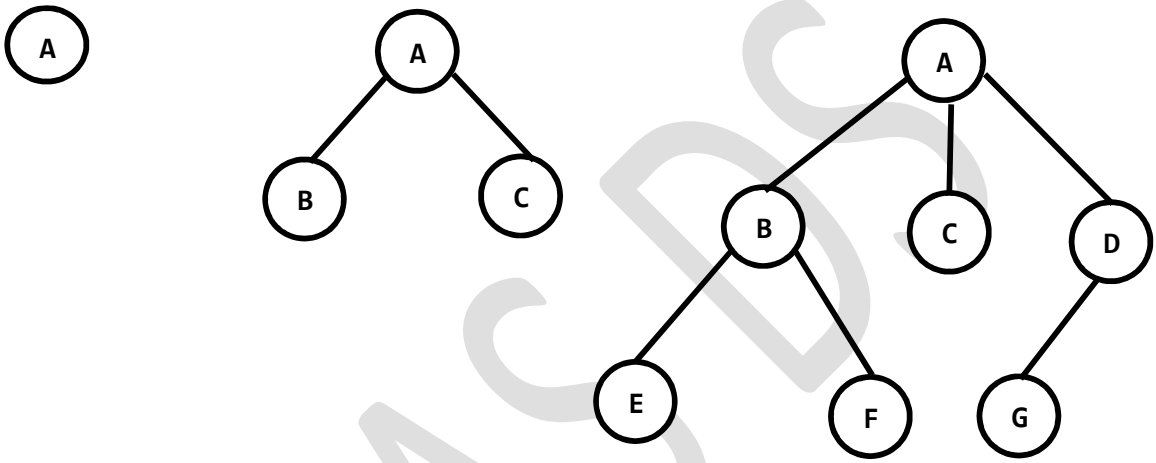
A binary Tree is one type of tree data structure, where each node has a maximum of two children and a minimum of zero children.

The child's name is the left child and the right child.

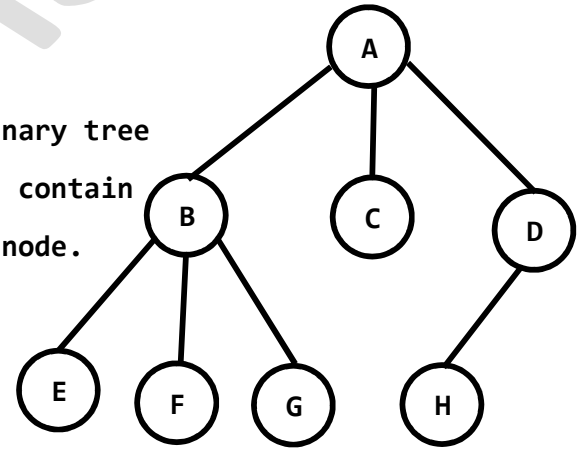
Properties of Binary Tree:-

1. **Maximum number of nodes on level i of a binary tree is 2^i. Example:-Root level is 0 (zero), so the number of nodes at root is 2^0=1**
2. **There is only one path from root to any node.**
3. **Tree with 'N' nodes has exactly 'N-1' edges connecting these nodes.**

Example:- All are binary tree except last.



This is not binary tree because B node contain more than two node.



Example:-1 Build a Binary Tree.

```
package com.ds;
public class BT
{
      static int index=-1;
      class Node{
            int data;
            Node left;
            Node right;
```

VIKAS DS

```
            Node(int data){
                  this.data = data;
                  this.left = null;
                  this.right = null;
            }
      }
      Node buildTree(int[] nodes){
            index++;
            if(nodes[index]==-1)
                  return null;
            Node node = new Node(nodes[index]);
            node.left = buildTree(nodes);
            node.right = buildTree(nodes);
            return node;
      }
      public static void main(String[] args)
      {

          int[] nodes = {1, 2, 4, -1, -1, 5, -1, -1, 3, -1, 6, -1, -
1};
          BT obj = new BT();
            System.out.println(obj.buildTree(nodes).data);//1
            //System.out.println(obj.buildTree(nodes).left.data);
            //System.out.println(obj.buildTree(nodes).right.data);
      }

}
```

Traverse data from Binary Tree:-

There are four ways to traverse the data from a binary Tree.

1. **Pre Order:-In this we traverse the data in the following way ROOT_NODE----LEFT_NODE----RIGHT_NODE.**

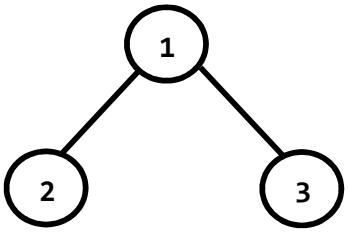
2. **In Order:- In this we traverse the data in the following way LEFT_NODE----ROOT_NODE----RIGHT_NODE.**

3. **Post Order:- In this we traverse the data in the following way LEFT_NODE----RIGHT_NODE----ROOT_NODE.**

4. **Level Order:-In this we traverse the data based on the level of the tree.**

VIKAS DS

Example:-



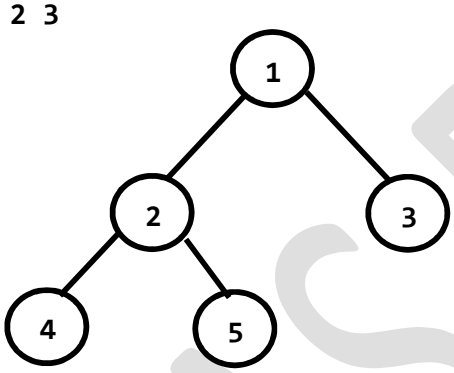
Pre Order:-1-2-3

In Order:-2-1-3

Post Order:-2-3-1

Level Order:-1

2 3



Pre Order:-1-2-4—5-3

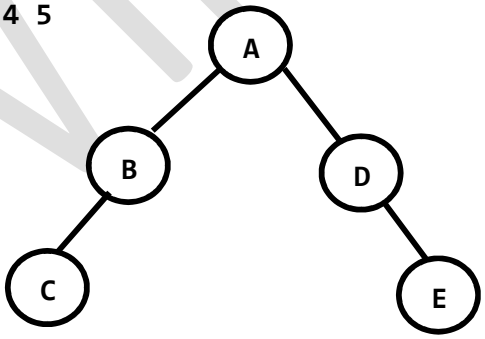
In Order:-4-2-5-1-3

Post Order:-4-5-2-3-1

Level Order:-1

2 3

4 5



Pre Order:-A-B-C-D-E

In Order:-C-D-A-D-E

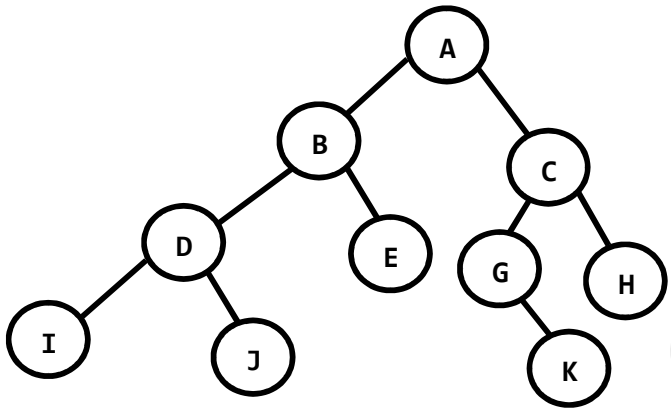
Post Order:-C-B-E-D-A

Level Order:-A

VIKAS DS

B D

C E



Pre Order:-A-B-D-I-J-E-C-G-K-H

In Order:-I-D-J-B-E-A-G-K-C-H

Post Order:-I-J-D-E-B-K-G-H-C-A

Level Order:-A

B C

D E G H

I J K

Example:-Traverse the data from the Binary Tree.

Node.java

```
package com.ds;
public class Node
{
int data;
Node left;
Node right;
Node(int data){
this.data = data;
this.left = null;
this.right = null;
}
}
```

BTTraverse.java

```
package com.ds;
public class BTTraverse
{
Node root;
```

VIKAS DS

```
public BTTraverse()
{
root = null;
}

void preOrder(Node node)
{
if(node==null)
return;
System.out.print(node.data+" ");
preOrder(node.left);
preOrder(node.right);
}
void inOrder(Node node)
{
if(node==null)
return;
inOrder(node.left);
System.out.print(node.data+" ");
inOrder(node.right);
}
void postOrder(Node node)
{
if(node==null)
return;
postOrder(node.left);
postOrder(node.right);
System.out.print(node.data+" ");
}
public static void main(String[] args)
{
BTTraverse obj = new BTTraverse();
obj.root = new Node(1);
obj.root.left = new Node(2);
obj.root.right = new Node(3);
obj.root.left.left = new Node(4);
obj.root.left.right = new Node(5);
obj.root.right.right = new Node(6);
System.out.print("InOrder Traversal ===> ");
obj.inOrder(obj.root);
System.out.println();
System.out.print("PreOrder Traversal ===> ");
obj.preOrder(obj.root);
System.out.println();
System.out.print("PostOrder Traversal ===> ");
obj.postOrder(obj.root);
System.out.println();
}
}
```

VIKAS DS

Result:-

```
InOrder Traversal ===> 4 2 5 1 3 6
PreOrder Traversal ===> 1 2 4 5 3 6
PostOrder Traversal ===> 4 5 2 6 3 1
```

Example:-Traverse the data in Level Order.

Node.java

```
package com.ds;

public class Node
{
int data;
Node left;
Node right;
Node(int data){
this.data = data;
this.left = null;
this.right = null;
}

}
```

BTLavelOrder.java

```
package com.ds;
import java.util.LinkedList;
import java.util.Queue;
public class BTLavelOrder
{
Node root;
public BTLavelOrder()
{
root = null;
}
public static void levelOrderTraversal(Node node){
if(node==null)
return;
Queue<Node> q = new LinkedList<>();
q.add(node);
q.add(null);
while(!q.isEmpty()){
Node cur = q.remove();
if(cur==null){
System.out.println();
if(q.isEmpty())
```

VIKAS DS

```
break;
else
q.add(null);
}
else{
System.out.print(cur.data+" ");
if(cur.left!=null)
q.add(cur.left);
if(cur.right!=null)
q.add(cur.right);
}
}
}
public static void main(String[] args)
{
BTLavelOrder obj = new BTLavelOrder();
obj.root = new Node(1);
obj.root.left = new Node(2);
obj.root.right = new Node(3);
obj.root.left.left = new Node(4);
obj.root.left.right = new Node(5);
obj.root.right.right = new Node(6);
System.out.println("Level Order Traversal ");
levelOrderTraversal(obj.root);
}
}
```

Result:-

```
Level Order Traversal
1
2 3
4 5 6
```

Example:-In this application we get the number of nodes, the sum of all nodes, the height of the tree, and search for a particular node.

Node.java

```
package com.ds;
public class Node
{
int data;
Node left;
Node right;
Node(int data){
this.data = data;
this.left = null;
this.right = null;
```

VIKAS DS

```
}

}
```

NodeCount.java

```
package com.ds;
public class NodeCount
{
Node root;
public NodeCount()
{
root = null;
}
public static int countNodes(Node node)
{
if(node==null)
return 0;
int ln = countNodes(node.left);
int rn = countNodes(node.right);
return ln+rn+1;
}
public static int sumOfNodes(Node node){
if(node==null)
return 0;
int ls = sumOfNodes(node.left);
int rs = sumOfNodes(node.right);
return ls+rs+node.data;
}
public static int height(Node node){
if(node==null)
return 0;
int lh = height(node.left);
int rh = height(node.right);
return Math.max(lh,rh)+1;
}

public static boolean search(Node node,int data){
if(node==null)
return false;
if(node.data == data)
return true;
if(search(node.left,data))
return true;
if(search(node.right,data))
return true;
return false;
}

public static void main(String[] args)
{
```

VIKAS DS

```
NodeCount obj=new NodeCount();
obj.root = new Node(1);
obj.root.left = new Node(2);
obj.root.right = new Node(3);
obj.root.left.left = new Node(4);
obj.root.left.right = new Node(5);
obj.root.right.left = new Node(6);
obj.root.right.right = new Node(7);
System.out.println("Number of Node="+countNodes(obj.root));
System.out.println("Sum of Node="+sumOfNodes(obj.root));
System.out.println("Height of Tree="+height(obj.root));
System.out.println("7 is found ="+search(obj.root,7));
System.out.println("11 is found ="+search(obj.root,11));
}
}
```

Result:-

```
Number of Node=7
Sum of Node=28
Height of Tree=3
7 is found =true
11 is found =false
```

VIKAS DS